

SIMULATION TOOL

Course Name: Digital Circuits

Course Code: ECA0202

Name: S.Yuvan Raju

Register No: 192512385

AIM

To analyze the effect of clock skew in a synchronous sequential circuit using simulation

PROCEDURE

1. Launch Xilinx Vivado and create a new RTL project (e.g. counter_comparison), selecting 'Do not specify sources at this time' and choosing any target part (e.g. xc7a35tcpg236-1), since the design is only meant for simulation.
2. Write the Verilog design source for the 4-bit Synchronous Counter (sync_counter.v), in which all flip-flops are triggered by the same clock edge, so all output bits change simultaneously.
3. Write the Verilog design source for the 4-bit Asynchronous (Ripple) Counter (async_counter.v), in which each flip-flop is clocked by the output of the previous stage, causing propagation delay to accumulate from one bit to the next.
4. Write a testbench (tb_counter_compare.v) that instantiates both counters and drives them with an identical clock (10 ns period) and an identical active-low reset signal, ensuring both circuits are compared under exactly the same clock conditions.
5. Add sync_counter.v and async_counter.v under Design Sources, and tb_counter_compare.v under Simulation Sources in the Vivado Sources panel.
6. Since tb_counter_compare is the only file under Simulation Sources, Vivado automatically sets it as the simulation top module.
7. Run Behavioral Simulation from the Flow Navigator (Simulation → Run Simulation → Run Behavioral Simulation). Vivado elaborates and compiles the design and opens the waveform viewer.
8. In the Objects panel, add clk, rst_n, sync_count[3:0] and async_count[3:0] to the waveform window. Expand the U_ASYNC instance to also observe the internal flip-flop outputs q0, q1, q2, and q3.
9. Run the simulation for the full test duration and use Zoom Fit followed by Zoom In to inspect individual clock edges, comparing how sync_count and async_count update relative to the clock.
10. Observe and record the differences: the synchronous counter changes all bits simultaneously on the clock edge, while the asynchronous counter shows a visible ripple/staggered update as each bit toggles the next stage in sequence.

VERILOG CODE

Synchronous Counter (sync_counter.v)

```
module sync_counter (
    input wire clk,
    input wire rst_n,
    output reg [3:0] count
);
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        count <= 4'b0000;
    else
        count <= count + 1'b1;
    end
endmodule
```

Asynchronous (Ripple) Counter (async_counter.v)

```
module async_counter (
    input wire clk,
    input wire rst_n,
    output wire [3:0] count
);
reg q0, q1, q2, q3;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) q0 <= 1'b0;
    else q0 <= #1 ~q0;
end
always @(posedge q0 or negedge rst_n) begin
    if (!rst_n) q1 <= 1'b0;
    else q1 <= #1 ~q1;
end
always @(posedge q1 or negedge rst_n) begin
    if (!rst_n) q2 <= 1'b0;
    else q2 <= #1 ~q2;
end
always @(posedge q2 or negedge rst_n) begin
    if (!rst_n) q3 <= 1'b0;
    else q3 <= #1 ~q3;
end

assign count = {q3, q2, q1, q0};
endmodule
```

Testbench (tb_counter_compare.v)

```
`timescale 1ns/1ps

module tb_counter_compare;
    reg clk = 0;
    reg rst_n = 0;
    wire [3:0] sync_count;
    wire [3:0] async_count;

    sync_counter U_SYNC (.clk(clk), .rst_n(rst_n), .count(sync_count));
    async_counter U_ASYNC (.clk(clk), .rst_n(rst_n), .count(async_count));

    always #5 clk = ~clk; // 100 MHz, identical clock to both counters

    initial begin
        rst_n = 0;
    end
endmodule
```

```
#12 rst_n = 1;
#300 $finish;
end

always @(posedge clk) begin
#2;
if (sync_count != async_count)
$display("[%0t ns] MISMATCH: sync=%b async=%b",
$time, sync_count, async_count);
end
endmodule
```

SIMULATION OUTPUT

The design was simulated in Vivado 2026.1 using the built-in Vivado Simulator (XSim). Both counters were run under the identical 10 ns period clock and the identical reset sequence. The following screenshots show the project setup, source hierarchy, and the resulting waveform comparison.

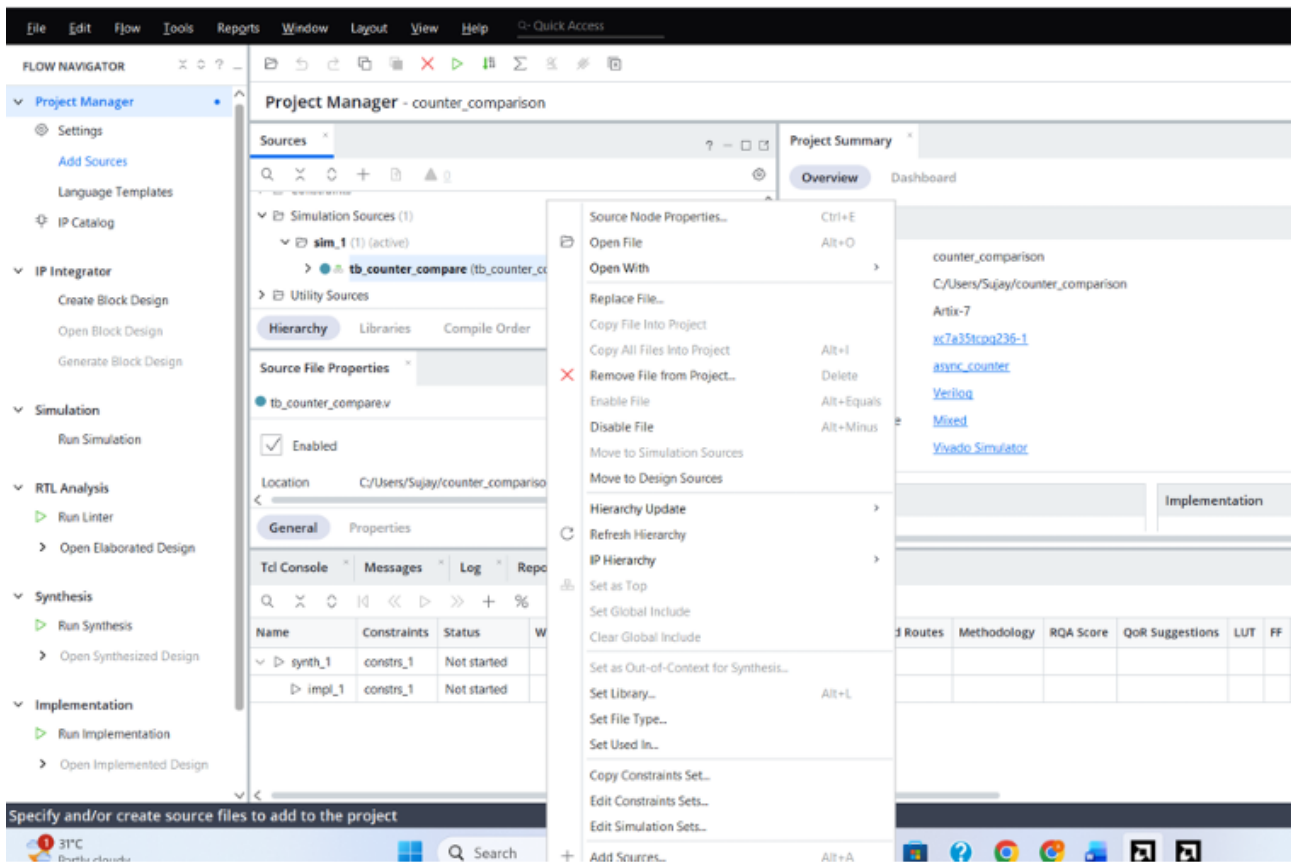


Fig. 1: Vivado project with sync_counter.v, async_counter.v and tb_counter_compare.v added, with the testbench set as simulation top.

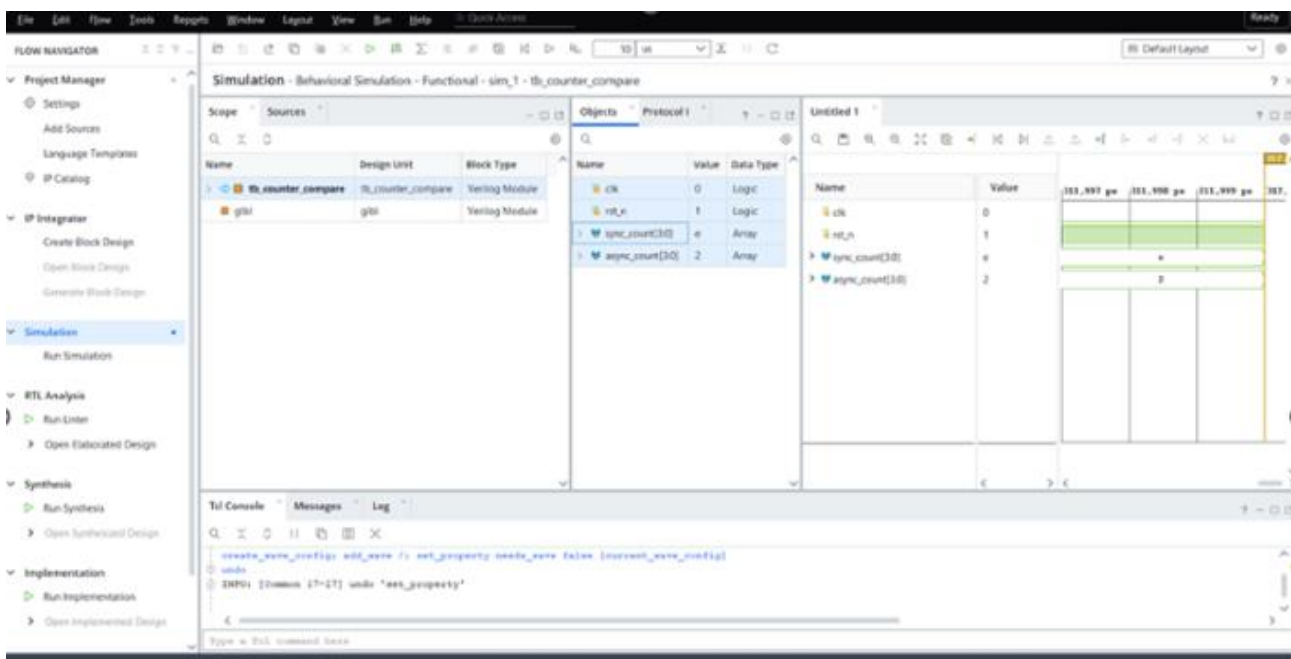


Fig. 2: Behavioral simulation launched successfully; XSim design snapshot loaded and simulation ran for 1000 ns.

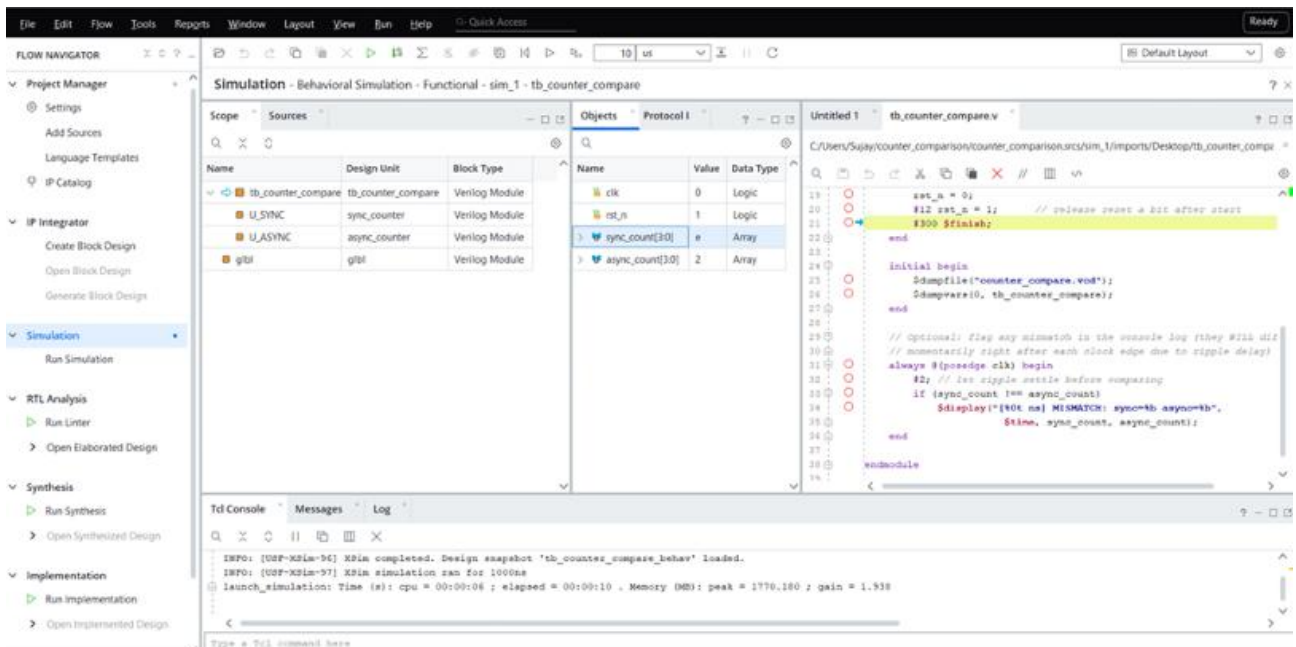


Fig. 3: Signals (clk, rst_n, sync_count, async_count) added to the waveform Objects panel prior to reopening the Waveform view.

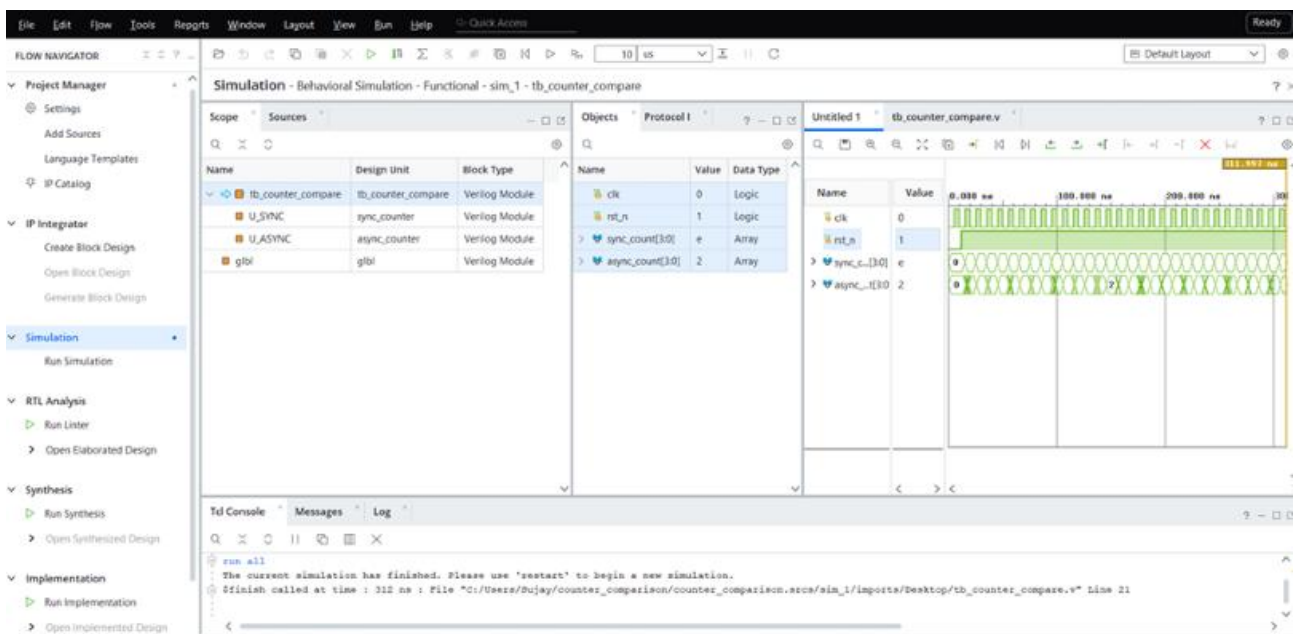


Fig. 4: Full waveform view after Zoom Fit, showing clk toggling, rst_n releasing at 12 ns, and both counters incrementing over the 300 ns run (\$finish called at 312 ns).

Observation

In the synchronous counter, all four output bits update simultaneously on every rising edge of the clock, since every flip-flop shares the same clock input. In the asynchronous (ripple) counter, only the least significant bit (q0) is directly clocked; each subsequent bit is triggered by the toggling of the previous bit, so the higher-order bits change with a small cumulative propagation delay after the clock edge. This was modeled using a 1 ns delay per stage in the ripple counter, and the testbench's mismatch-detection block confirmed that `sync_count` and `async_count` briefly disagree immediately after each clock edge before the ripple settles, whereas the synchronous counter output is stable and glitch-free at every clock cycle.

RESULT

The synchronous and asynchronous 4-bit counters were successfully designed and simulated in Vivado under identical clock conditions. The waveform results confirm that the synchronous counter changes state cleanly and instantaneously on each clock edge, while the asynchronous counter exhibits a ripple delay across its stages, verifying the expected theoretical difference between the two counter architectures.